UNIVERSITÉ TOULOUSE CAPITOLE

Academic Year 2026–2027
Master MIAGE I2S

REPORT

Software Design

Assignment 1

Group members:
Nguyen Hoang Nhan LE
Hanane ELOUCHEFOUNE

Professor:
Prof. Laurent Perrussel

Submission date: 30/10/2026

Contents

1 Use Case Diagrams **4**
1.1 Identification of use cases and actors 4
1.2 Use case diagram 6
1.3 Details of the "Pickup Car" use case 7
1.3.1 System sequence diagram 7
1.3.2 Low-fidelity interfaces 7

2 Class Diagram **14**
2.1 Design choices 14
2.2 Class diagram 16

3 Sequence Diagram **17**
3.1 Reservation process 17
3.1.1 Overview 17
3.1.2 Describe the reservation and calculate the price 18
3.1.3 Identify the customer 19
3.1.4 Finalise the reservation 20

List of Figures

1.1 General use case diagram 6
1.2 System sequence diagram for the Pickup Car use case 7
1.3 Main screen of the low-fidelity interface for the pickup process 8
1.4 Reservation details for the pickup process 9
1.5 Select and allocate an available car in the booked category 10
1.6 Check the driving licence and store its scan 11
1.7 Record the joint inspection and the customer's digital signature 11
1.8 Record payment for a pay-at-pickup reservation 12
1.9 Review completed checks before handing over the keys 13

2.1 Class diagram 16

3.1 Reservation overview 17
3.2 Describe the reservation and calculate the price 18
3.3 Identify the customer 19
3.4 Finalise the reservation 20

List of Tables

1.1 Actors in the system 4
1.2 Use cases and their participating actors 4

Chapter 1

Use Case Diagrams

1.1 Identification of use cases and actors

Based on the application description, we identified the following main actors.

Actor	**Justification**
Customer	As mentioned in the assignment: "...customer starts a reservation by mentioning where they want to pick up the car..."
Agency Employee	As mentioned in the assignment: "Based on the chosen category, the agency employee will allocate a specific car to the customer."
Hotline Operator	As mentioned in the assignment: "The 24/7 hotline will contact her back and organize with her the repairing actions"

Table 1.1: Actors in the system

Design choices: Each actor represents a role interacting with the system. We chose to use a single Customer actor before and after signing in because authentication changes the customer's access to the system without changing their role. The Agency Employee actor represents an employee working at an agency location who handles pickup and return activities. The Hotline Operator actor handles incident reports and coordinates assistance following an incident.

These actors participate in the following use cases:

Use case	**Participating actors**
Make Reservation	Customer
Pay	Customer
Cancel Reservation	Customer
Pickup Car	Customer, Agency Employee
Return Car	Customer, Agency Employee
Manage Incident	Customer, Hotline Operator
Report Incident	Customer

Table 1.2: Use cases and their participating actors

The `Make Reservation` use case covers selecting the pickup location, rental dates, car category, insurance and extra options. It also covers signing in and providing customer

details before completing the booking. We keep these activities within one use case because they contribute to the same customer goal. We do not represent each form or selection as a separate use case.

We separate `Pay` from the other use cases because payment can occur during reservation, cancellation or pickup. We use conditional *extend* relationships to show when payment is required. `Pay` extends `Make Reservation` when prepayment is selected. It extends `Cancel Reservation` when a prepaid reservation requires a cancellation fee. It extends `Pickup Car` when prepayment was not selected and the rental payment must be settled at the agency. These extensions are conditional, but payment is required when the corresponding condition applies.

The `Cancel Reservation` use case allows the customer to cancel an existing booking. We assume that the customer must sign in and identify their reservation before cancellation. According to the assignment, cancelling a prepaid booking incurs a cancellation fee. The cancellation payment therefore represents this fee rather than another payment of the full rental price.

The `Pickup Car` use case involves both the customer and the agency employee. It covers finding the reservation, allocating a car from the booked category, checking and scanning the driving licence, and recording the pickup inspection with the customer's digital signature. Any required rental payment must also be completed before the keys are handed over. We keep these checks within the pickup use case and describe them in more detail through the system sequence diagram and low-fidelity interfaces.

The `Return Car` use case also involves the customer and the agency employee. The employee checks the car's condition, records the mileage and the return date and time, and obtains the customer's signature on the evaluation. The car is returned to the pickup location, as required. We do not model additional charges for damage, fuel or late returns because they are not mentioned in the assignment.

We separate `Report Incident` from `Manage Incident` because they have different goals. Reporting requires the customer to provide the location, date, time and circumstances of an accident or breakdown, together with at least one photo. Managing the incident covers the follow-up between the hotline operator and the customer to organise assistance. This separation allows the report to be recorded before the hotline begins its response.

We treat signing in as part of the relevant use cases or as a precondition because authentication supports the rental process without constituting a separate business goal.

1.2 Use case diagram

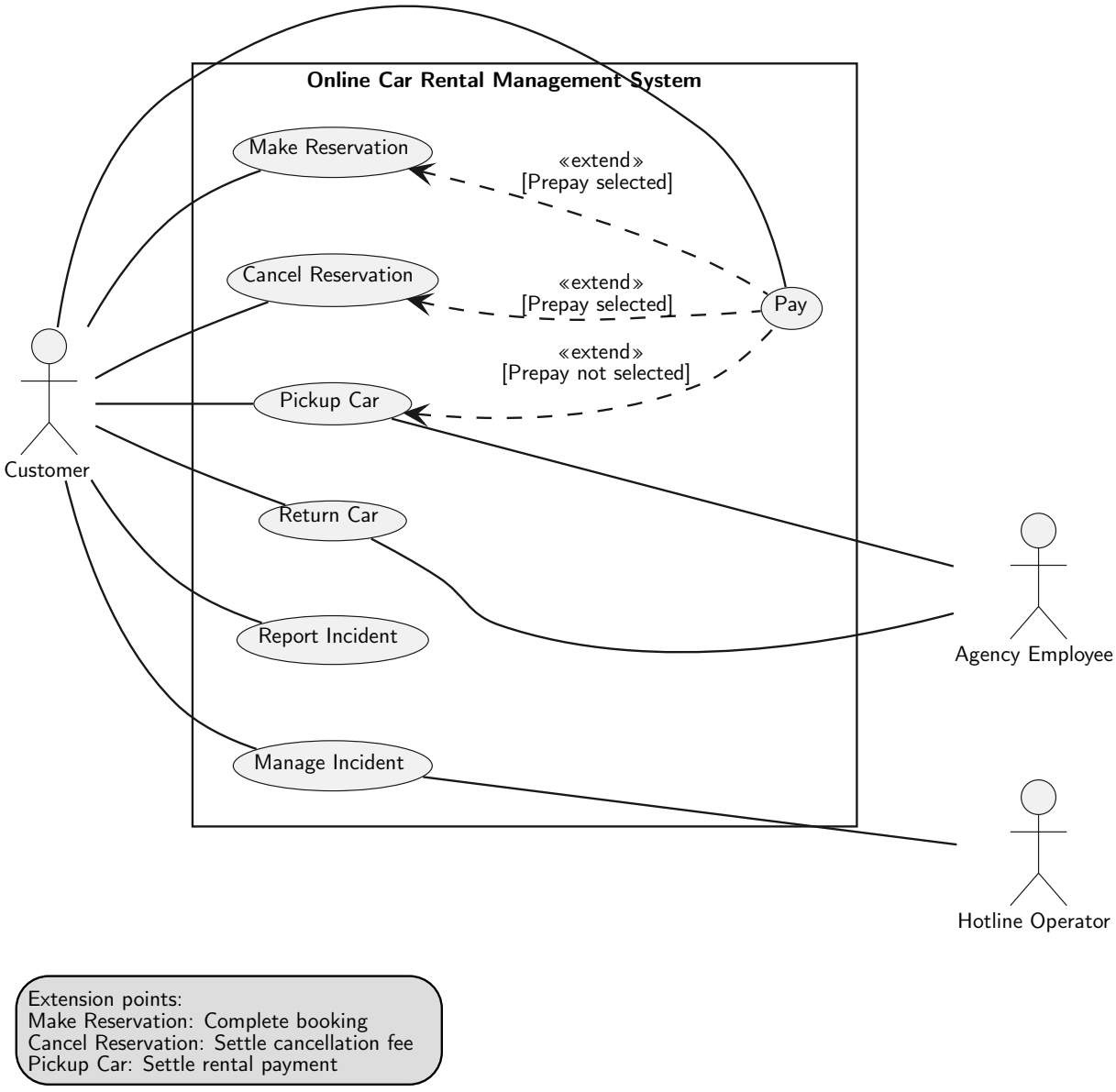


Figure 1.1: General use case diagram

1.3 Details of the “Pickup Car” use case

1.3.1 System sequence diagram

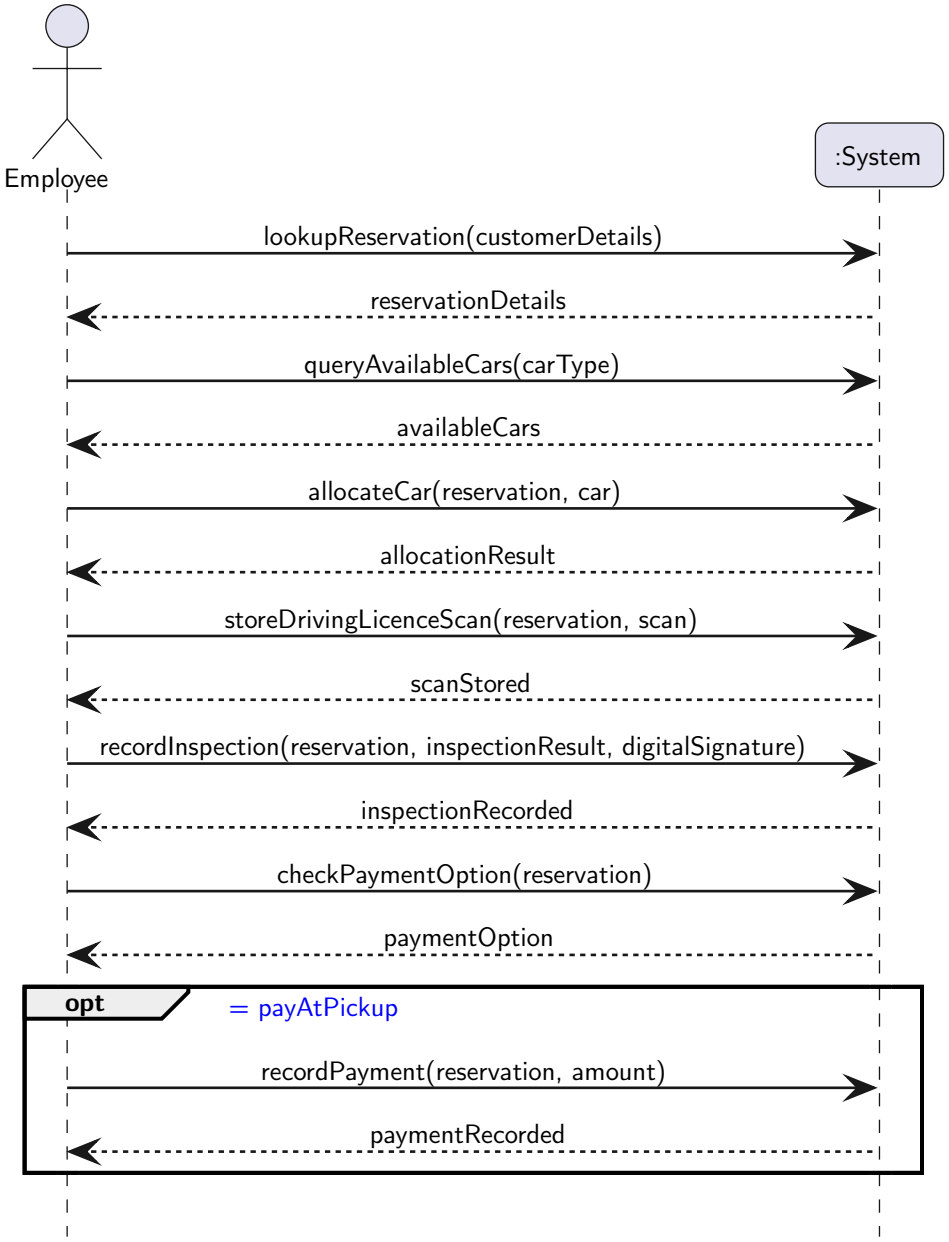


Figure 1.2: System sequence diagram for the Pickup Car use case

1.3.2 Low-fidelity interfaces

We have designed low-fidelity interfaces for the pickup process. The activities follow this sequence: find the reservation, review its details, allocate a car from the booked category, check and scan the driving licence, and record the inspection results with the customer’s digital signature. The navigation buttons link to the corresponding figures in this PDF to illustrate each step of the workflow. Each action must succeed before moving to the

next step. The keys are handed over only after all checks and any required payment are complete.

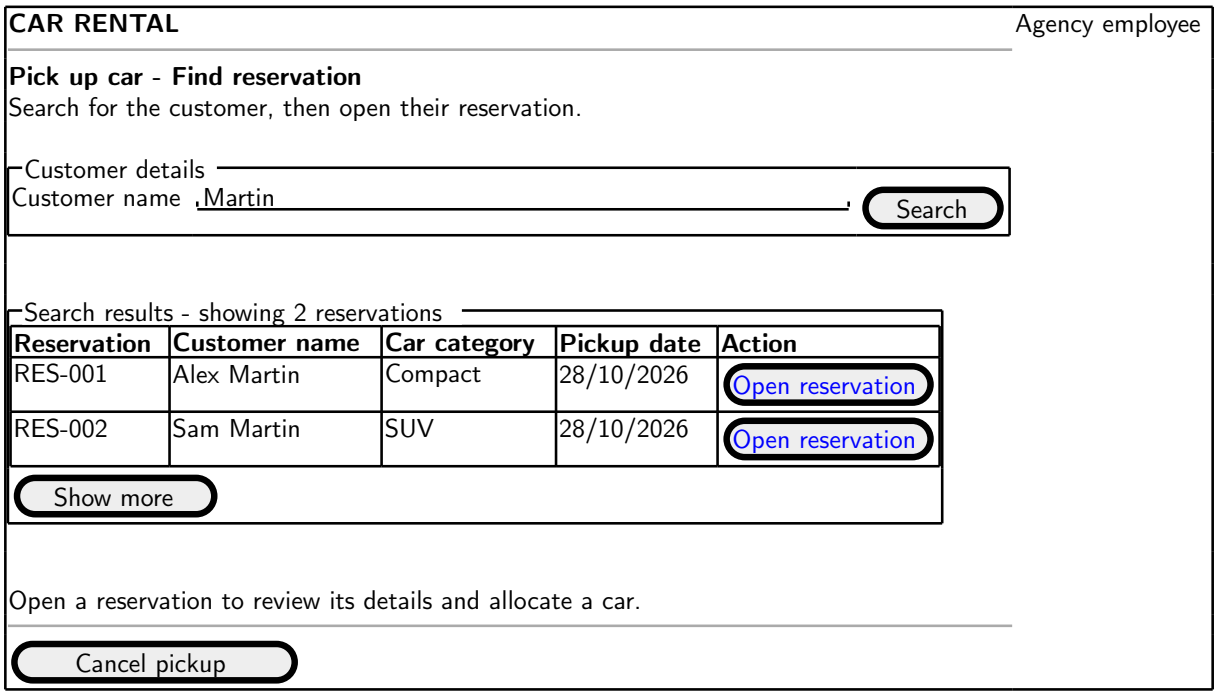


Figure 1.3: Main screen of the low-fidelity interface for the pickup process

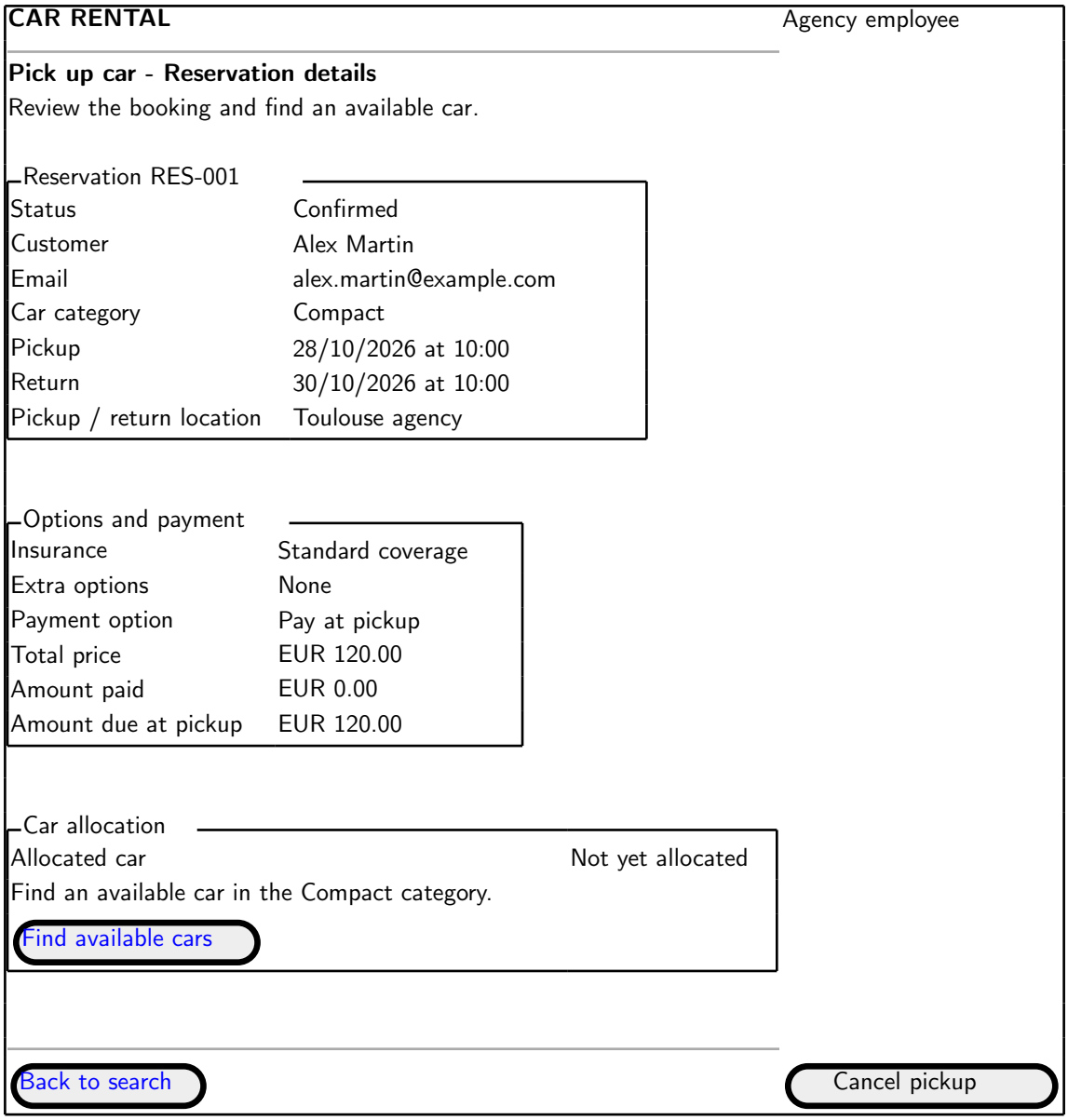


Figure 1.4: Reservation details for the pickup process

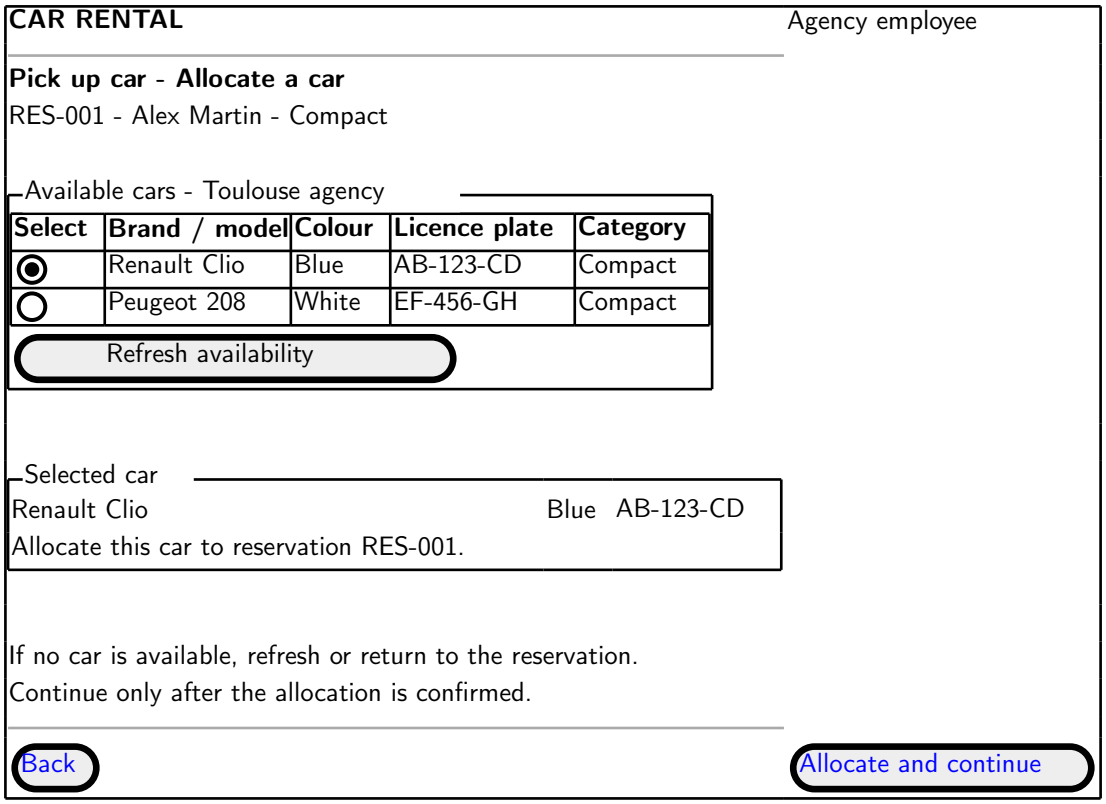


Figure 1.5: Select and allocate an available car in the booked category

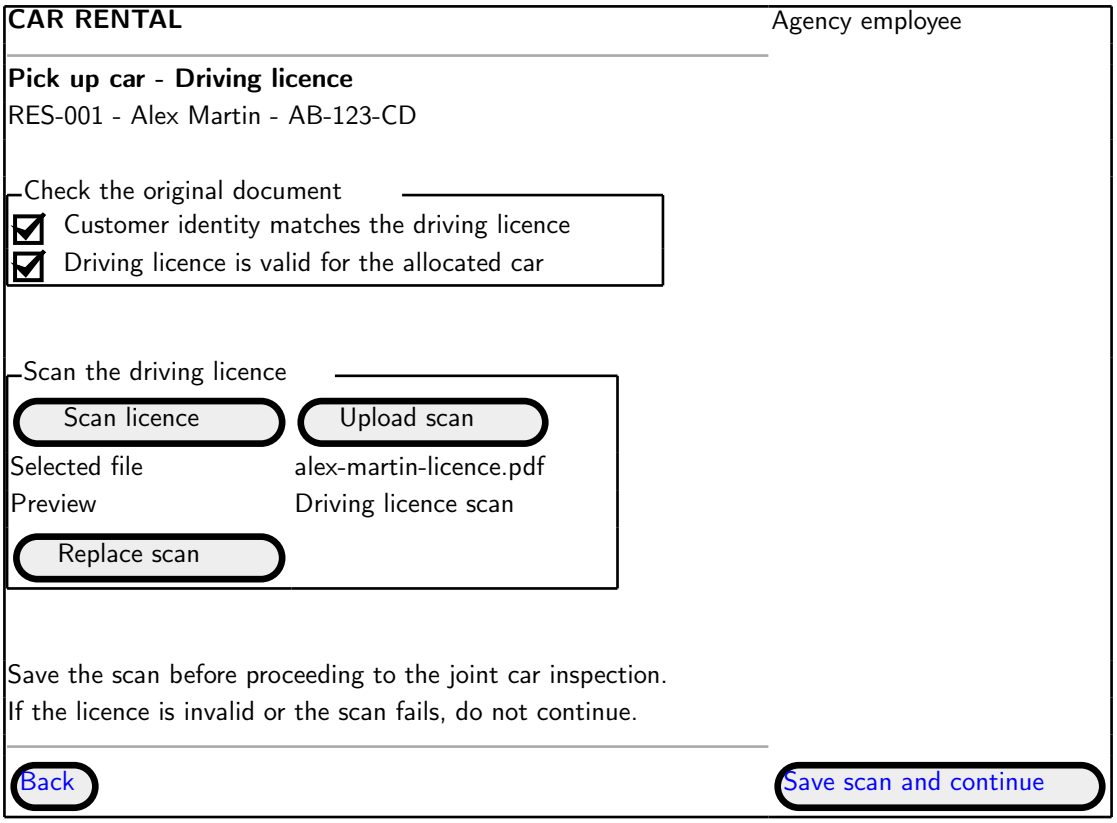


Figure 1.6: Check the driving licence and store its scan

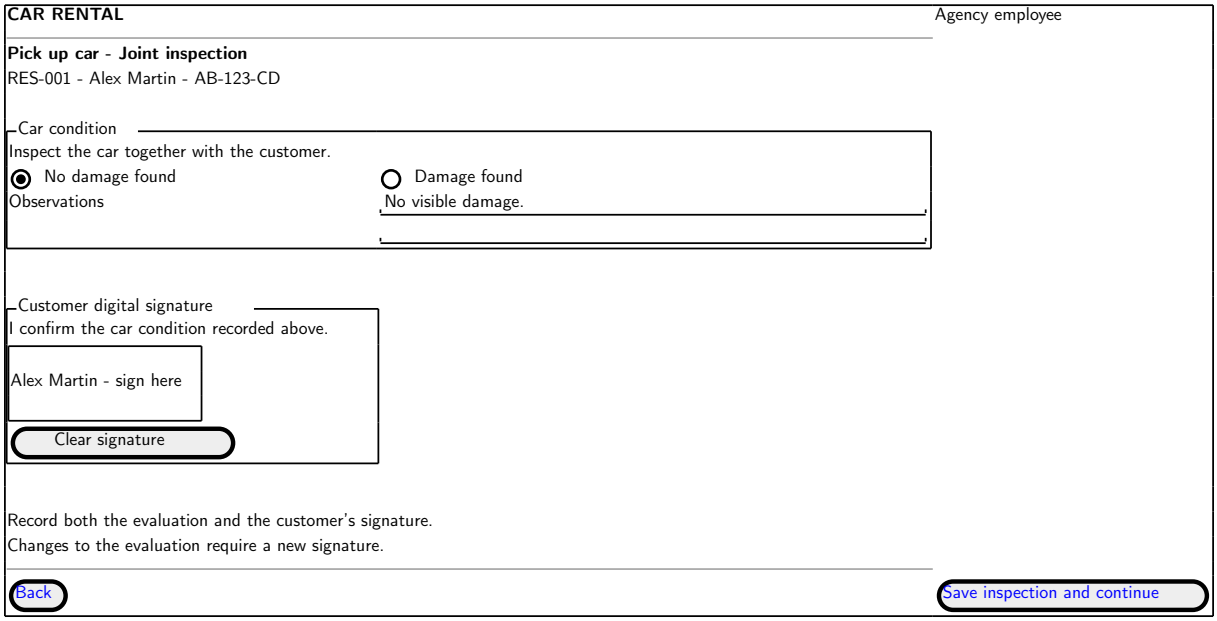


Figure 1.7: Record the joint inspection and the customer's digital signature

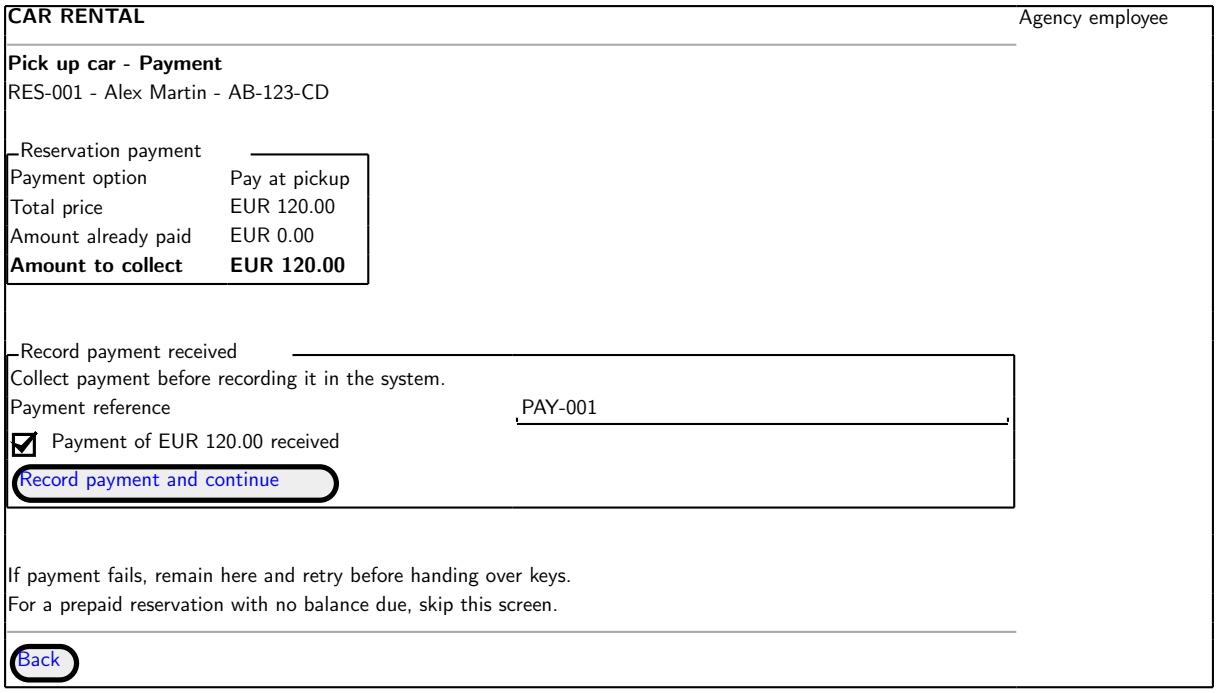


Figure 1.8: Record payment for a pay-at-pickup reservation

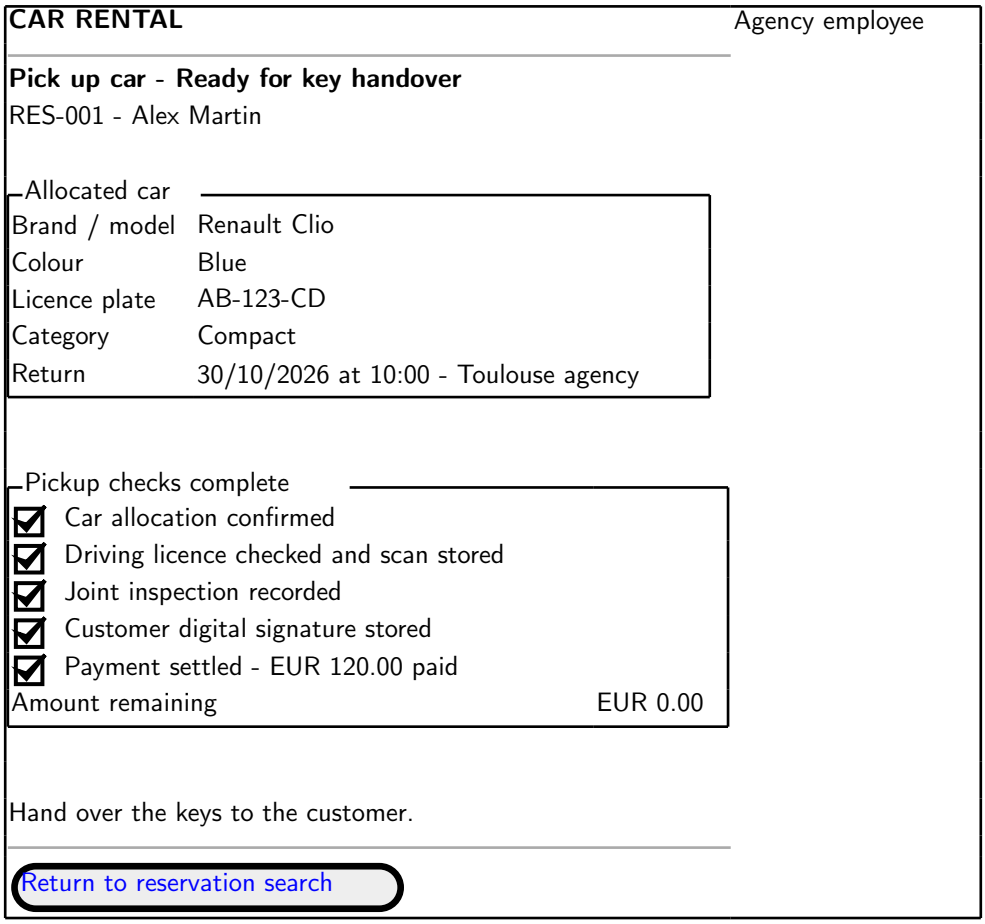


Figure 1.9: Review completed checks before handing over the keys

Chapter 2

Class Diagram

2.1 Design choices

We have designed the class diagram to represent the main business entities of the car rental application. The classes follow the rental process: the customer chooses a car category and extra services, makes a reservation, pays either in advance or at pickup, picks up a car, and returns it after use. The diagram focuses on the business information and relationships. Interface components and controllers are not included in this high-level view.

The `Customer` class stores the customer's email address, first name and last name. We use a separate `Address` class to group the street, city, country and postal code. We assume that each customer has one recorded address, while several customers can share the same address. We also use `Address` to represent the pickup and return location instead of introducing a separate agency class. The customer's home address and the rental location are different objects of the same class.

The `DriverLicense` class stores a link to a scanned driving licence. A customer can have multiple recorded licences, each represented by a separate object. A customer may have no recorded licence when the reservation is made. The employee checks and scans the relevant licence during pickup.

We separate `CarType` from `Car` because the customer books a category before a specific vehicle is allocated. `CarType` describes the category through its name, description, typical numbers of passengers and windows, gearbox type and example car. `Car` stores the brand, colour and licence plate number of an actual vehicle. Each car belongs to one category, and one category can contain several cars.

We model `Reservation` as an abstract class with two concrete subclasses. The `PrepaidReservation` subclass represents prepaid bookings, while `PaylaterReservation` represents bookings paid at pickup. They share attributes but differ in behaviour, particularly when a reservation is cancelled.

We also separate `Reservation` from `Rental`. The reservation records the planned dates, payment option, status and agreed price. The rental records the actual pickup and return information, including the return mileage. A reservation can exist without a rental if the customer has not collected the car or has cancelled the booking. Each rental uses one specific car. The same car can be used for several rentals over time, but we assume that these rentals do not overlap.

The `Insurance` and `ExtraOption` classes describe the services associated with a reservation. Insurance has a daily price, while an extra option has a price and a picture. We

keep these classes separate because their information and pricing are different. The same insurance or option can be selected in several reservations. Included insurance should not add an extra charge, although the current diagram does not explicitly distinguish included coverage from additional coverage.

The `Payment` class records each payment's amount, date, time and purpose. Payment options are represented by the `PaymentOption` enumeration, which distinguishes prepayment from payment at pickup. We do not create separate payment subclasses. Both choices use the same payment information; the main differences are the payment timing and pricing rules. A reservation can have several payments, for example, a rental payment followed by a cancellation fee. It can also have no recorded payment yet when payment at pickup is selected.

The `PickupVehicleInspection` and `ReturnVehicleInspection` classes record the car's condition and the customer's digital signature. Each rental must have one pickup inspection and may have one return inspection, depending on its status. Both inspections must be signed before the rental is completed. The signature can remain empty while an inspection is being prepared.

The `Incident` class records an accident or breakdown during a rental. It stores the location, date and time, and circumstances. Each incident must have at least one photo, represented by an `IncidentPhoto` object. A rental can have no incident or several incidents. We use enumerations for reservation status, payment option, payment purpose, inspection type, incident type and gearbox type to keep these values consistent.

We use composition between `Rental` and `Incident`, and between `Incident` and `IncidentPhoto`, to express exclusive ownership: each incident belongs to one rental, and each photo belongs to one incident. Their lifecycles depend on their respective owners. The other relationships use regular associations. We use `Integer` for counts and mileage, `Decimal` for monetary values, and `String` for licence plates and postal codes because they may contain letters or leading zeros.

In `Rental`, `returnDateTime` and `returnMileage` are optional because the actual return time and mileage are unknown until the car is returned.

2.2 Class diagram

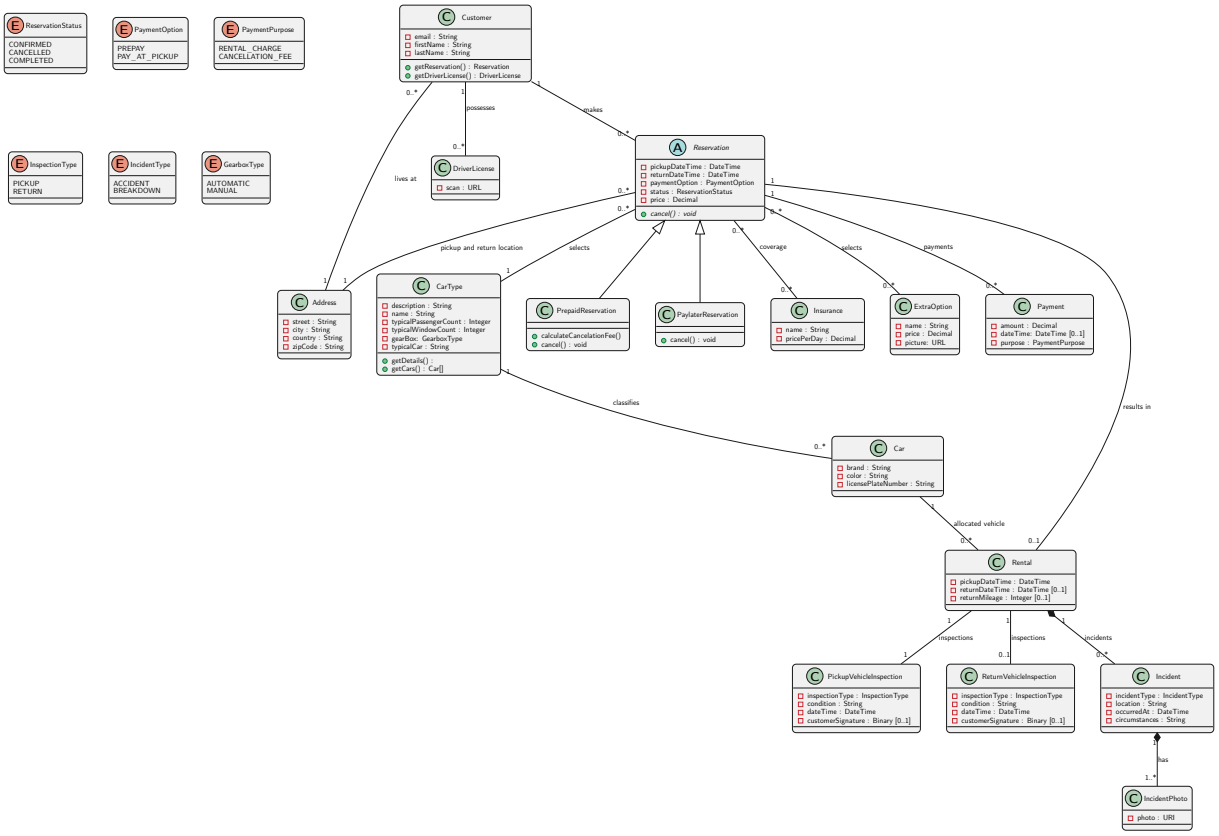


Figure 2.1: Class diagram

Chapter 3

Sequence Diagram

3.1 Reservation process

3.1.1 Overview

The reservation process allows a customer to book a rental car for a selected location and period. We divide the customer's journey into three stages.

First, the customer describes their needs, and the system presents available offers to help them choose a car category, insurance, extra options and a payment option.

Second, the customer signs in and provides their email, first name, last name and address.

Finally, the customer confirms the booking. If prepayment is selected, the customer pays by credit card, and the payment is recorded in the system. A confirmed reservation is then created.

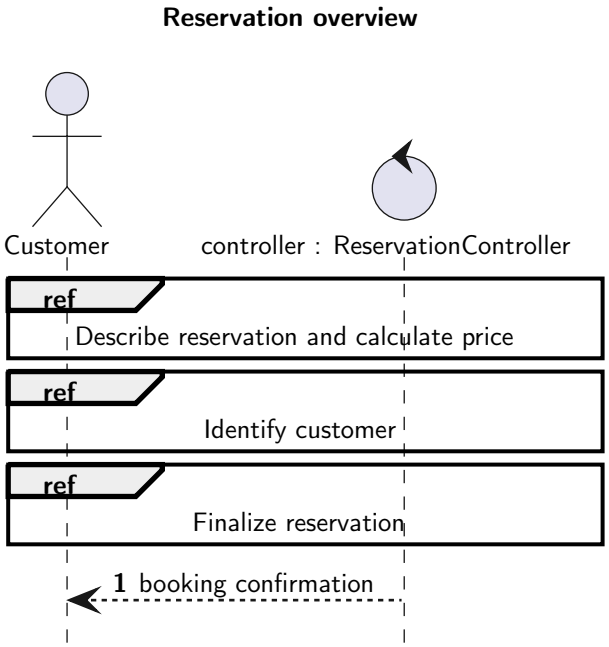


Figure 3.1: Reservation overview

3.1.2 Describe the reservation and calculate the price

The customer provides the pickup location and rental dates to search for offers. The `ReservationController` looks up available offers based on this information. The customer then chooses a `CarType`. The controller obtains the `CarType` details and presents the included insurance and available extras. The customer then selects the desired extras. For each selected additional insurance policy and extra option, the controller retrieves the price from the corresponding `Insurance` or `ExtraOption` object. Once the customer chooses a payment option, the controller calculates the quote and returns a selection summary.

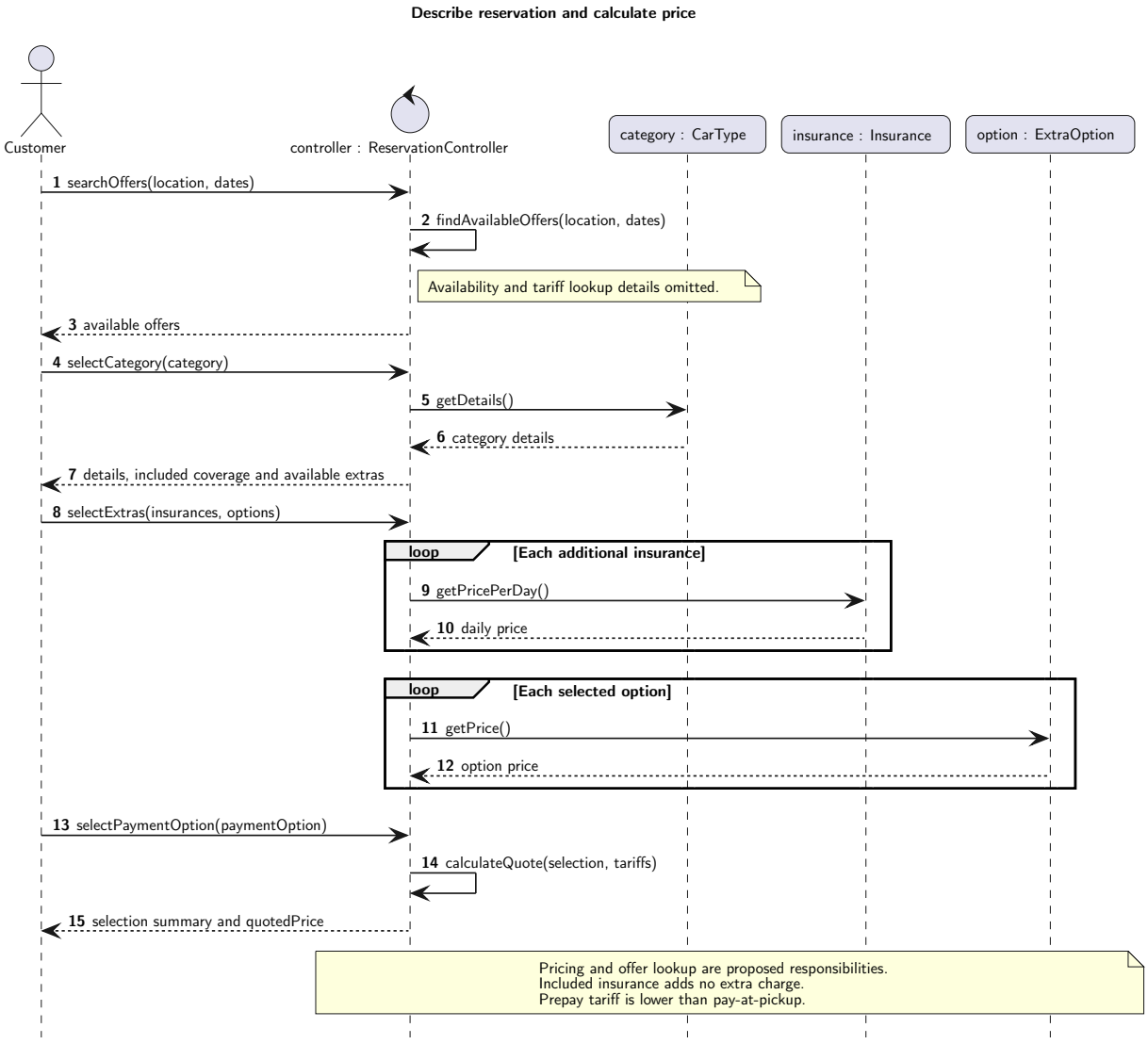


Figure 3.2: Describe the reservation and calculate the price

3.1.3 Identify the customer

The customer must sign in before completing the reservation. The controller authenticates the customer and retrieves the corresponding `Customer` object. After successful authentication, the customer provides their email, first name, last name and address. The controller updates the customer's details and indicates that the booking is ready for confirmation. The customer's address is distinct from the pickup location. We decided not to include authentication infrastructure details in this diagram.

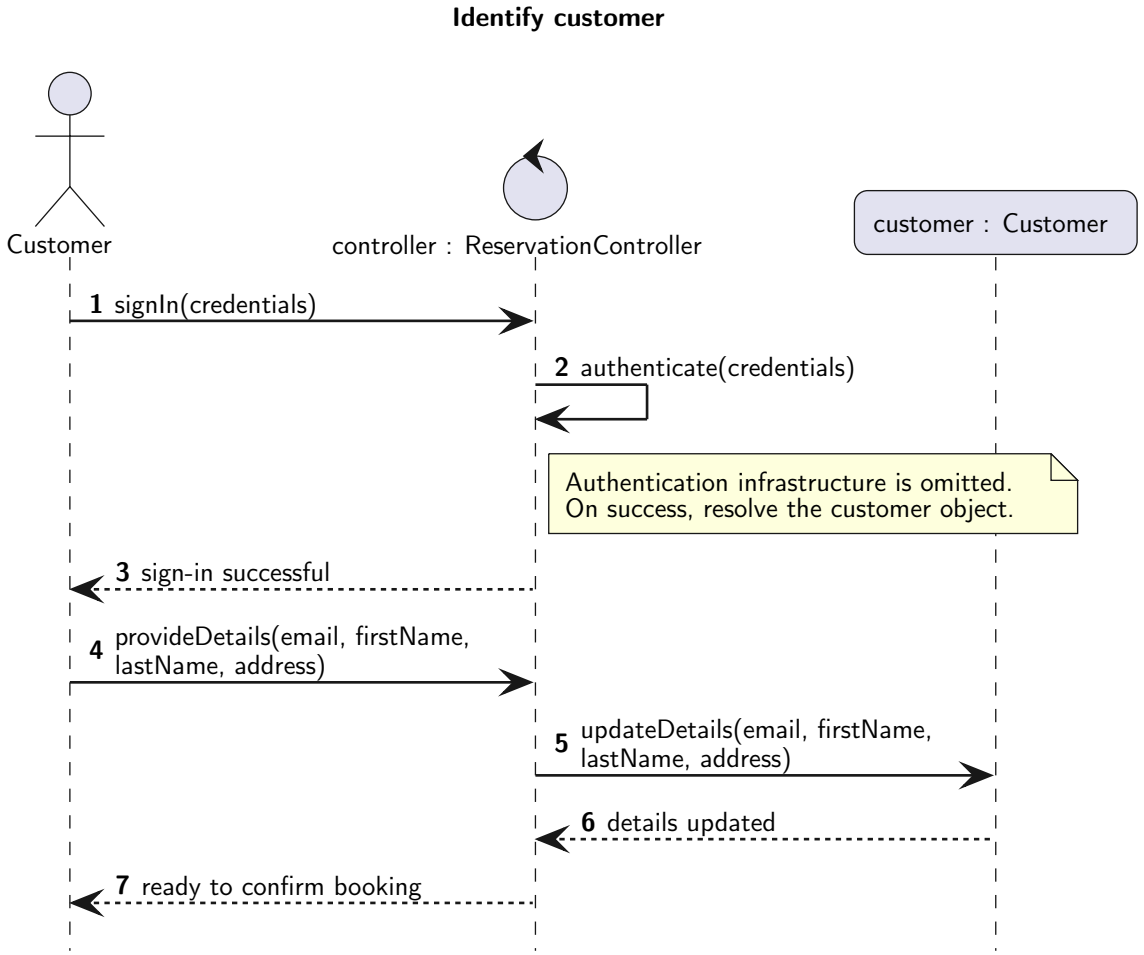


Figure 3.3: Identify the customer

3.1.4 Finalise the reservation

The customer confirms that the booking details are correct. The controller rechecks availability and reserves capacity for the selected car category. The *alt* fragments distinguish between the two payment options. In the case of prepayment, the controller creates a `Payment` object only after successful payment is verified. For payment at pickup, no `Payment` object is created at this stage. The controller then creates a confirmed `PrepaidReservation` or `PaylaterReservation`, depending on the payment option. The reservation contains the selected dates, category, location, insurance, extra options, payment option and quoted price. For prepayment, the controller associates the payment with the reservation. The resulting confirmation is returned to the controller and delivered to the customer, as shown in the overview diagram.

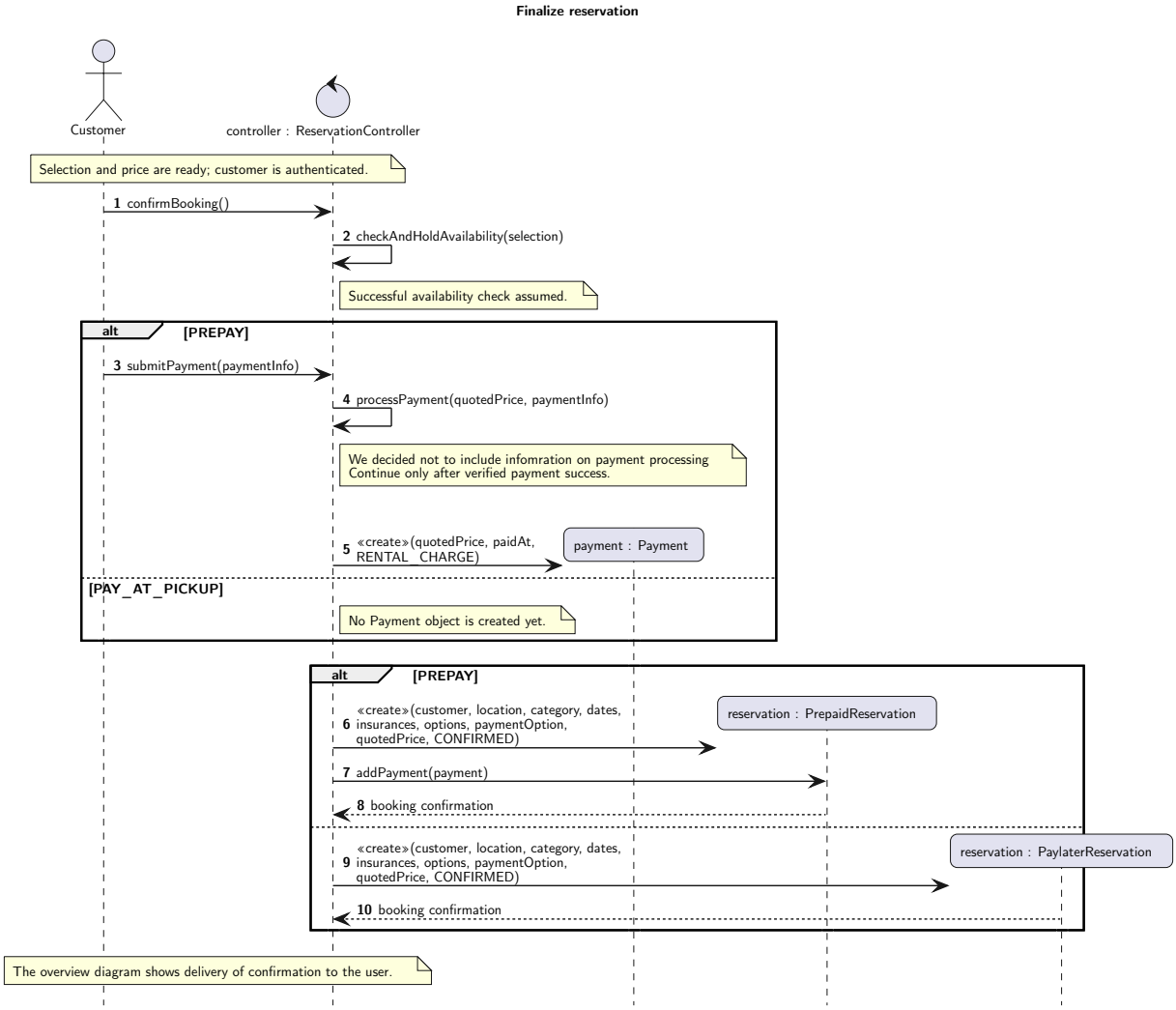


Figure 3.4: Finalise the reservation